USER MANUAL

Q_{10} Calculator for Circadian Period

Browser, desktop (Python) and Google Colab versions

Estimating the temperature coefficient Q_{10} and the Arrhenius activation energy of a circadian clock from period measurements at several temperatures.

$$\tau(T) = \tau_b \cdot Q_{10}^{\frac{T_b - T}{10}}$$

Covers: index.html (browser), q10_calculator.py (desktop), circadian_period_q10.ipynb (Google Colab)

Repository: github.com/dxda6216/q10

Web app: dxda6216.github.io/q10

License: MIT License, © 2026 dxda6216

Manual date: October 2026

Contents

1 Introduction **3**

1.1 What the calculator does 3

1.2 Three versions 3

1.3 Which version should I use? 3

1.4 Conventions in this manual 4

2 Background **5**

2.1 The temperature coefficient Q_{10} 5

2.2 Temperature compensation of circadian clocks 5

2.3 The model that is fitted 5

2.4 The base temperature 6

2.5 The Arrhenius plot and its relation to Q_{10} 6

2.6 The Arrhenius regression line 7

3 Calculation method **8**

3.1 Objective 8

3.2 Starting values 8

3.3 Optimisation 8

3.4 Standard errors 8

3.5 Goodness of fit 8

3.6 Fitted curve 9

3.7 Arrhenius regression 9

3.8 Agreement between the versions 10

3.9 Optional polynomial fit 10

4 Preparing your data **11**

4.1 Requirements 11

4.2 Data formats 11

4.3 Replicates and averages 11

4.4 Choosing a temperature range 12

4.5 Choosing a base temperature 12

5.1 Getting and opening the file 12

5.2 Screen layout 13

5.3 Step by step 14

5.4 Setting the fit range 15

5.5 Setting the axis scales 16

5.6 Polynomial fit (optional) 17

5.7 The plots 19

5.8 The report 19

5.9 Saving 20

5.10 The “How it works” window 20

5.11 Phones, tablets and dark mode 21

6 Python version **22**

6.1 Requirements 22
6.2 Installation 22
6.3 Anaconda and Miniconda 22
6.4 Starting the program 23
6.5 The window 23
6.6 Step by step 24
6.7 Saving 26

7 Google Colab version 27
7.1 Opening the notebook 27
7.2 Form fields 27
7.3 Running and output 28
7.4 Saving 28
7.5 Using the notebook in Jupyter 28

8 Output reference 29
8.1 Results text 29
8.2 Plots 29
8.3 Tab-delimited data 30

9 Worked example 31
9.1 All points, base temperature 37 °C 31
9.2 Fit range 28–40 °C 31
9.3 Polynomial fit (browser and Python) 31
9.4 Reading the example 32

10 Interpreting the results 33

11 Error messages and troubleshooting 34
11.1 Error messages 34
11.2 Troubleshooting 35

12 Privacy and data handling 36

13 Differences between the versions 38

14 Use of AI 39
14.1 Statement from the AI assistant 39
14.2 How the results were checked 40

15 License 41
15.1 What this means in practice 41

Appendix A Formula summary 42

Appendix B Glossary 43

Appendix C Third-party components 44

1 Introduction

1.1 What the calculator does

The Q_{10} calculator estimates how strongly the period of a circadian rhythm depends on temperature. You give it period measurements made at several temperatures; it fits the standard Q_{10} model to the data and reports:

- the temperature coefficient $\mathbf{Q_{10}}$ with its standard error;
- the **estimated period** $\boldsymbol{\tau_b}$ at a base temperature of your choice, with its standard error;
- the coefficient of determination $\mathbf{R^2}$ of the fit;
- an **Arrhenius regression** of ln(frequency) on 1000/T, giving the **activation energy** $\mathbf{E_a}$, the slope and intercept with standard errors, and R^2.

The results are shown as three plots (period, frequency and Arrhenius) and as a text report that you can save or paste into a spreadsheet. In the browser and Python versions you can also choose the temperature range for the fit simply by dragging across a plot; the fit is recalculated at once, while the axis scales stay those of the all-data fit so that results for different ranges can be compared directly. The axis scales are set automatically, but can also be set manually in all three versions. Optionally, the browser and Python versions also fit a polynomial of period on temperature and report its equation and parameters.

1.2 Three versions

The same calculator is available in three versions. They use the same model, the same fitting method and the same inputs, and produce the same numbers (see Section 3.8).

	Browser (HTML)	Desktop (Python)	Google Colab
File	`index.html`	`q10_calculator.py`	`circadian_period_q10.ipynb`
Runs on	Your computer, in a web browser	Your computer	Google's servers
Installation	None	Python with numpy, SciPy and matplotlib	None (a Google account is needed)
Internet	Not needed once the file is downloaded	Not needed	Required
Data leaves your computer	No	No	Yes: processed by Google Colab
Saving	PNG, SVG figure; TXT report	PNG, PDF, SVG figure; TXT results	Save image; copy text

1.3 Which version should I use?

- **Browser version** — the easiest choice for most people. Use it online at dxda6216.github.io/q10 or as a downloaded file. Nothing to install, works on any computer, phone or tablet with a modern browser, and keeps your data on your own device. Good for unpublished data and for sharing the tool with colleagues.

- **Python version** — for people who already use Python, need a PDF figure, want to zoom into the plots, or want to read and adapt the code.
- **Colab version** — when you want to run the calculation without downloading anything, or prefer a notebook. Note that your data is processed on Google's servers.

1.4 Conventions in this manual

Names of buttons, fields and tabs are printed like **Calculate**. Keys are shown as Ctrl+Enter; on a Mac use ⌘ instead of Ctrl in the browser version. Temperatures are in degrees Celsius (°C) unless stated otherwise; T in the Arrhenius equations is the absolute temperature in kelvin (K = °C + 273.15). Periods are in hours, and frequencies in cycles per day (day^{-1}). All numerical examples use the example data built into the calculator (Section 9).

2 Background

2.1 The temperature coefficient Q_{10}

Q_{10} is the factor by which the rate of a process changes when the temperature rises by 10 °C. If the rate is R_1 at temperature T_1 and R_2 at T_2,

$$Q_{10} = \left(\frac{R_2}{R_1}\right)^{\frac{10}{T_2 - T_1}}$$

For example, $Q_{10} = 2$ means the rate doubles for every 10 °C of warming. Most enzyme-catalysed reactions have Q_{10} of about 2 to 3.

2.2 Temperature compensation of circadian clocks

For a circadian clock, the natural "rate" is its frequency: the number of cycles per unit time, which is the reciprocal of the period. A defining property of circadian clocks is **temperature compensation**: although the underlying biochemistry speeds up with temperature, the period stays nearly constant over the physiological range. Q_{10} of a circadian period is therefore typically close to 1, roughly between 0.8 and 1.2.

Value	Meaning
$Q_{10} > 1$	The period shortens as temperature rises: the clock runs faster (under-compensation).
$Q_{10} = 1$	The period does not change with temperature: perfect compensation.
$Q_{10} < 1$	The period lengthens as temperature rises: the clock runs slower (over-compensation).

2.3 The model that is fitted

Applying the definition of Q_{10} to frequency (rate = 1/period) and writing τ_b for the period at a reference or *base* temperature T_b gives the model fitted by the calculator:

$$\tau(T) = \tau_b \cdot Q_{10}^{\frac{T_b - T}{10}} \tag{1}$$

The frequency in cycles per day is

$$f(T) = \frac{24}{\tau(T)}$$

and taking logarithms turns the model into a straight line in T with slope $\ln(Q_{10})/10$:

$$\ln f(T) = \ln \frac{24}{\tau_b} + \frac{T - T_b}{10} \ln Q_{10} \tag{2}$$

Equation (2) is why a plot of ln(frequency) against temperature would be a straight line if the model held exactly. The calculator, however, fits equation (1) directly to the periods (see Section 3.1).

2.4 The base temperature

The base temperature T_b does not change the model; it only decides *at which temperature* the fitted period τ_b is reported. Changing T_b is a reparameterisation, so:

- Q_{10}, its standard error, R^2 and the fitted curve are identical for every choice of base temperature;
- only τ_b and its standard error change.

With the example data, for instance, τ_b is 23.662 ± 0.221 h at 37 °C and 28.840 ± 0.232 h at 30 °C, while Q_{10} = 1.327 ± 0.019 in both cases.

> A base temperature far outside your data — absolute zero (−273.15 °C) is offered as an option — makes τ_b a long extrapolation of the model, with a correspondingly large standard error. Use such values only if you specifically need them.

2.5 The Arrhenius plot and its relation to Q_{10}

The Arrhenius equation describes the temperature dependence of a rate constant k through an activation energy E_a:

$$k = A \cdot \exp\left(-\frac{E_a}{RT}\right)$$

where A is a pre-exponential factor, R = 8.314 J mol^{-1} K^{-1} is the gas constant and T is the absolute temperature. Taking logarithms gives a straight line in 1/T with slope $-E_a/R$:

$$\ln k = \ln A - \frac{E_a}{R} \cdot \frac{1}{T}$$

The calculator's Arrhenius plot shows ln(frequency) against 1000/T, with T in kelvin. Points that fall on a straight line in this plot follow the Arrhenius equation; the steeper the line, the larger E_a.

How E_a and Q_{10} are related

Both descriptions predict the frequency ratio between two temperatures T_1 and T_2 (in kelvin). From equation (2) and from the Arrhenius equation respectively,

$$\ln\frac{f_2}{f_1} = \frac{T_2 - T_1}{10}\ln Q_{10} \quad \text{and} \quad \ln\frac{f_2}{f_1} = \frac{E_a}{R}\left(\frac{1}{T_1} - \frac{1}{T_2}\right) \tag{3}$$

Setting the two expressions equal, and noting that $1/T_1 - 1/T_2 = (T_2 - T_1)/(T_1T_2)$, gives

$$E_a = \frac{RT_1T_2\ln Q_{10}}{10} \quad \text{equivalently} \quad Q_{10} = \exp\left(\frac{10E_a}{RT_1T_2}\right) \tag{4}$$

This conversion is exact for a single pair of temperatures. Over a range of temperatures it is only approximate, because the two models have different shapes: the Q_{10} model has a constant slope in T, the Arrhenius model a constant slope in 1/T. Their local slopes agree when

$$\frac{d\ln f}{dT} = \frac{\ln Q_{10}}{10} \quad (Q_{10}\text{model}) \qquad \frac{d\ln f}{dT} = \frac{E_a}{RT^2} \quad (\text{Arrhenius})$$

so the same E_a corresponds to a slightly different Q_{10} at each temperature. Across a typical range of 15–20 °C the difference is small.

2.6 The Arrhenius regression line

In addition to the fitted Q_{10} curve, the calculator can draw the ordinary least-squares straight line of ln(frequency) on 1000/T in the Arrhenius plot. It is fitted to the same data points used for Q_{10}:

$$\ln f = b_0 + b_1 \cdot \frac{1000}{T} \qquad E_a = -1000 \cdot R \cdot b_1 \tag{5}$$

This line is shown by default (solid grey) and can be switched off. Its E_a, slope b_1, intercept b_0, standard errors and R^2 are always included in the results.

Why the two lines are close but not identical

1. **Different model forms.** The Q_{10} model is a straight line in T (equation 2); the Arrhenius model is a straight line in 1/T. Plotted against 1000/T, the Q_{10} curve is therefore slightly bent.
2. **Different residuals.** The Q_{10} fit minimises the squared errors of the *period*; the Arrhenius regression minimises the squared errors of *ln(frequency)*. The two criteria weight the data points slightly differently.

Within the range of the data the two lines nearly coincide — for the example data their largest difference is about 0.005 in ln(frequency), roughly 0.5 % in frequency and about one sixth of the scatter of the data about the line. Outside the data range, where both are extrapolations, they separate slightly. The E_a from the regression and the E_a converted from Q_{10} with equation (4) are also very close: 21.86 and 22.0 kJ/mol for the example data.

3 Calculation method

3.1 Objective

The parameters τ_b and Q_{10} are estimated by non-linear least squares on the periods themselves (not on their logarithms), minimising the sum of squared residuals

$$\mathrm{SSE} = \sum\nolimits_i \left(\tau_i - \widehat{\tau}\left(T_i \right) \right)^2$$

where τ_i is the measured period at temperature T_i and $\hat{\tau}(T_i)$ the period predicted by equation (1). All data points have equal weight. Only the points selected for the fit (all points, or those inside the chosen temperature range) are used.

3.2 Starting values

Non-linear fitting needs starting values. The calculator obtains them from a straight-line regression of ln(period) on temperature: by equation (2) its slope s and intercept c correspond to $\ln Q_{10} = -10s$ and $\ln \tau_b = c - (T_b/10)\cdot \ln Q_{10}$. These values are already close to the final solution, which makes the fit fast and reliable. If this step fails, the calculator starts from $\tau_b = 24$ h and $Q_{10} = 1$.

3.3 Optimisation

The Python and Colab versions use SciPy's `curve_fit` (for problems without bounds it uses the Levenberg–Marquardt algorithm of MINPACK). The browser version contains its own implementation of the Levenberg–Marquardt algorithm in JavaScript, written for this calculator. At each iteration it solves

$$\left(J^T J + \lambda \cdot \mathrm{diag}\left(J^T J \right) \right) \delta = J^T r$$

for the parameter step δ, where J is the Jacobian of the model with respect to the parameters and r the vector of residuals. After a step that lowers SSE, the damping factor λ is reduced (moving towards the Gauss–Newton method); otherwise it is increased (moving towards steepest descent) and the step is retried. The browser version iterates on $\ln \tau_b$ and $\ln Q_{10}$, which keeps both parameters positive; this does not change the objective, so it converges to the same minimum.

3.4 Standard errors

Standard errors are taken from the covariance matrix of the parameters,

$$\mathrm{Cov} = s^2 \left(J^T J \right)^{-1}, \qquad s^2 = \frac{\mathrm{SSE}}{n-2}$$

where J is evaluated at the solution with respect to τ_b and Q_{10}, and n is the number of points used in the fit. The ± values in the results are the square roots of the diagonal elements. This is the same definition that `curve_fit` uses by default (`absolute_sigma=False`): the errors are scaled by the observed scatter of the data about the fitted curve.

3.5 Goodness of fit

$$R^2 = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}}, \qquad \mathrm{SST} = \sum \left(\tau_i - \bar{\tau} \right)^2$$

$\bar{\tau}$ is the mean of the periods used in the fit. R^2 is the fraction of the variation in period that the model explains; see Section 10 for how to read it.

3.6 Fitted curve

The fitted curve is drawn with 200 evenly spaced points. Its temperature range covers all the data (including points outside a chosen fit range) and extends on each side by one third (33.3 %) of the data's temperature span; the end points are truncated to whole degrees. For the example data (25.1–41.1 °C) the curve runs from 19 to 47 °C. The same span is used for the Arrhenius regression line.

Axis scales

The axis limits of every plot are set to cover all data points and the curves fitted to *all* data, plus a margin of 5 % on each side. When a temperature range is chosen, the calculator still fits the all-data curves in the background for this purpose only, so the axis scales are identical whatever range is used. A range fit whose curve is steeper than the all-data fit may leave the plot area near its ends; the part outside is clipped at the frame. In the Arrhenius plot, the Arrhenius regression line is included in the limits only while it is shown.

Manual axis scales

Instead of the automatic limits, a temperature range (0–100 °C) and a period range (0–180 h) can be set by hand. They are used exactly as entered, without a margin, and are converted for the other plots:

Plot	x-axis from the temperature range [T_{min}, T_{max}]	y-axis from the period range [P_{min}, P_{max}]
Period	T_{min} to T_{max}	P_{min} to P_{max}
Frequency	T_{min} to T_{max}	$24/P_{max}$ to $24/P_{min}$
Arrhenius	$1000/(T_{max} + 273.15)$ to $1000/(T_{min} + 273.15)$	$\ln(24/P_{max})$ to $\ln(24/P_{min})$

Because $24/P$ is infinite for $P = 0$, a period minimum of 0 leaves the *upper* limit of the frequency and Arrhenius axes automatic. Points and curves outside manual limits are clipped at the frame. Either axis can be set on its own; the other stays automatic. The fitted curve is still drawn only over its usual span (one third beyond the data on each side), so it does not reach the edges of a much wider axis.

3.7 Arrhenius regression

For each data point used in the fit,

$$x_i = \frac{1000}{T_i + 273.15}, \qquad y_i = \ln\frac{24}{\tau_i}$$

and the regression line $y = b_0 + b_1 x$ is found by ordinary least squares:

$$b_1 = \frac{S_{xy}}{S_{xx}}, \qquad b_0 = \bar{y} - b_1\bar{x}$$

with $S_{xx} = \Sigma(x_i - \bar{x})^2$ and $S_{xy} = \Sigma(x_i - \bar{x})(y_i - \bar{y})$. With $s^2 = SSE/(n - 2)$ from the regression residuals, the standard errors are

$$\mathrm{SE}\left(b_1\right) = \sqrt{\frac{s^2}{S_{xx}}}, \qquad \mathrm{SE}\left(b_0\right) = \sqrt{s^2\left(\frac{1}{n} + \frac{\bar{x}^2}{S_{xx}}\right)}$$

and the activation energy follows from the slope:

$$E_a = -1000Rb_1, \qquad \mathrm{SE}\left(E_a\right) = 1000R \cdot \mathrm{SE}\left(b_1\right)$$

using R = 8.314462618 J mol^{-1} K^{-1}. E_a is reported in kJ/mol.

3.8 Agreement between the versions

The three versions were checked against each other with the example data, with several base temperatures (30 °C, 37 °C, Median and absolute zero) and with a restricted range (28–40 °C). τ_b, Q_{10}, their standard errors, R^2 and all Arrhenius regression values agree to every printed digit. The Arrhenius regression values also agree with SciPy's `linregress`.

3.9 Optional polynomial fit

When the polynomial option is switched on (browser and Python versions), an ordinary least-squares polynomial of period on temperature is fitted in addition to the Q_{10} model:

$$\tau\left(T\right) = a_0 + a_1T + a_2T^2 + \ \ldots\ + a_dT^d \tag{6}$$

with T in °C and τ in hours, degree d = 1 to 6, fitted to all data or to the data within its own temperature range. Powers of temperature grow very quickly ($41^6 \approx 5 \times 10^9$), so fitting them directly is numerically unreliable at higher degrees. The calculator therefore fits the polynomial in the scaled variable u = (T − c)/h, where c and h are the midpoint and half-width of the fitted temperatures (so u runs from −1 to 1), using a QR decomposition of the design matrix. The coefficients a_j in T are then obtained exactly by binomial expansion, and their covariance matrix is transformed in the same way, so the reported standard errors are those of a direct fit in T. With $s^2 = \mathrm{SSE}/(n - d - 1)$:

- standard errors: square roots of the diagonal of $s^2(U^TU)^{-1}$, transformed to the coefficients in T;
- $R^2 = 1 - \mathrm{SSE}/\mathrm{SST}$, adjusted $R^2 = 1 - (1 - R^2)(n - 1)/(n - d - 1)$, residual standard error = s.

At least d + 2 points are needed. The fitted curve is evaluated at 200 temperatures spanning all data plus 10 % of that span on each side. The results agree with NumPy's least-squares solution to rounding precision for degrees 1–5, and with an exact rational-arithmetic solution to about 10^{-12} for degree 6 (where a fit on raw powers in double precision is already inaccurate). The browser and Python versions agree to rounding precision for all degrees.

4 Preparing your data

4.1 Requirements

- Temperatures in **°C** and periods in **hours**.
- Exactly **one period for each temperature**.
- At least **3 data points** in the fit (two parameters are estimated, and standard errors need at least one degree of freedom). More points give more reliable standard errors.
- All periods must be **positive**.
- The same temperature may appear more than once (for example, for replicate measurements).

4.2 Data formats

Browser and Python versions: the data box

Type or paste one temperature–period pair per line. The two values can be separated by commas, spaces, tabs or semicolons, and these can be mixed. Blank lines and lines starting with `#` are ignored, and one header line at the top is skipped automatically, so data copied from a spreadsheet usually works as it is:

```
Temperature, Period
# Strain A, constant darkness
25.1, 33.6
26.9  32.8
27.3    30.4
```

Browser and Python versions: CSV files

Load CSV… reads a comma-, semicolon-, tab- or space-separated text file (.csv, .tsv or .txt). Temperature must be in the first column and period in the second; further columns are ignored. Rows whose first two cells are not both numbers (such as headers) are skipped. The rows read are written into the data box, so you can check and edit them before calculating.

Colab version: two lists

Enter the temperatures and the periods as two comma-separated lists in the same order:

```
Temperatures = 25.1, 26.9, 27.3, 29.2
Periods      = 33.6, 32.8, 30.4, 29.3
```

4.3 Replicates and averages

Every pair you enter counts as one independent observation with equal weight. You can enter either

- **individual measurements** — one line per animal, culture or recording, repeating the temperature as needed; or
- **mean periods** — one line per temperature.

Both are valid, but they answer slightly different questions and give different standard errors: with individual measurements, the standard errors reflect the scatter between replicates; with means, they reflect only the scatter of the means about the curve and are based on fewer points. State which you

used when you report results. The calculator does not apply weights, so means based on very different numbers of replicates are treated equally.

4.4 Choosing a temperature range

Circadian clocks are often well compensated within a physiological range and less so near its limits, where the period may change sharply or the rhythm may become unstable. The temperature-range option lets you fit Q_{10} to a chosen part of the data while still plotting all points. Points outside the range are shown in light green and are not used for any calculation, including the Arrhenius regression and the Minimum/Maximum/Average/Median base temperatures.

In the browser and Python versions the range can be set either by typing Low and High — after a first calculation the results update as you type — or by dragging across any plot (Sections 5.4 and 6.6). The Colab version uses two sliders.

The choice can make a real difference: with the example data, Q_{10} is 1.327 ± 0.019 using all 18 points but 1.261 ± 0.026 using the 13 points between 28 and 40 °C. Decide on the range on biological grounds, ideally before looking at the result, and always report it.

4.5 Choosing a base temperature

Choose the temperature at which a period estimate is most useful — often a standard culture or body temperature (for example 25 °C for many insects and fungi, or 37 °C for mammalian cells), or a value inside your data such as the median. Because the base temperature does not affect Q_{10} (Section 2.4), you can change it freely to read off the fitted period at different temperatures.

Option	Value used
−273.15 °C, 0, 4, 25, 30, 37, 100 °C	That fixed temperature.
Minimum, Maximum, Average, Median	Computed from the temperatures of the points used in the fit.
Custom value	Any temperature you enter (in Colab, set with the **Base_temperature** slider, 0–100 °C).

5.1 Getting and opening the file

Online

The easiest way is to open the web app at dxda6216.github.io/q10/index.html. The page is hosted on GitHub Pages, but the calculations run in your browser on your own computer; the data you enter and the results are not sent or stored anywhere (Section 12).

As a file on your computer

1. Open github.com/dxda6216/q10/blob/main/index.html.
2. Click **Download raw file** (the download icon above the file contents) and save `index.html`.
3. Double-click the saved file, or open it from your browser's **File → Open** menu.

No installation, server or account is needed; the downloaded file works without an internet connection and can be copied to other computers or sent to colleagues. Any current version of Chrome, Edge, Firefox or Safari works.

5.2 Screen layout

On a computer the page fits the browser window (from about 1280 × 800 pixels upwards) without scrolling: the input panel is on the left, and the results fill the rest of the window. The panel is kept compact so that the data box gets as much height as possible.

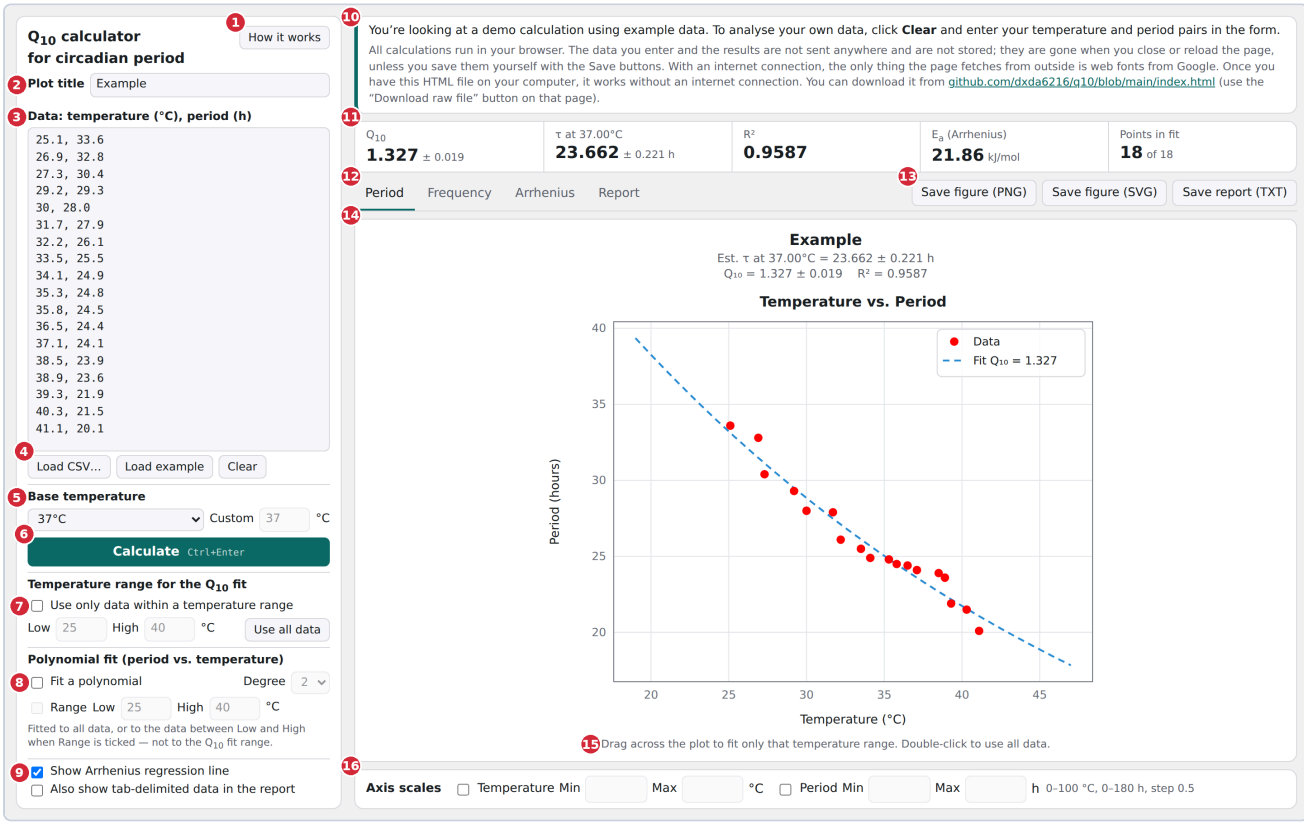


Figure 1. The browser version as it opens, showing the demo calculation. Numbers refer to the table below.

	Element	Purpose
1	**How it works**	Opens a window explaining the theory, the method, how to use the calculator and how your data is handled (Section 5.10).
2	**Plot title**	Optional title shown above the plots and in saved figures; also used for saved file names. Plots update as you type.
3	**Data box**	Temperature–period pairs, one per line (Section 4.2). It takes all the height the other controls leave free.
4	**Load CSV… / Load example / Clear**	Read data from a file, reload the example data, or clear the data, the title and all results.
5	**Base temperature**	Temperature at which τ_b is reported; the Custom field is enabled when "Custom value (enter at right)" is selected. Changing it recalculates at once.
6	**Calculate**	Runs the calculation (also Ctrl+Enter / ⌘+Enter). If something is wrong, a red message appears directly below the button; otherwise nothing is shown there, because the results appear in the summary row (11).
7	**Temperature range for the Q_{10} fit**	Tick to fit Q_{10} only to points with Low ≤ T ≤ High; editing Low or High recalculates at once. **Use all data** switches the range off.
8	**Polynomial fit**	Optional polynomial of period on temperature, with its own degree and range (Section 5.6). Off by default.
9	**Options**	Show the Arrhenius regression line; include tab-delimited data in the report.
10	**Notice**	Shown on opening: explains the demo and how your data is handled. Hidden by Clear, by editing the data or by loading a CSV.
11	**Summary**	Q_{10} and τ_b (each ± standard error), R^2, the Arrhenius activation energy E_a, and the number of points used in the fit.
12	**Tabs**	Switch between the Period, Frequency and Arrhenius plots and the text report.
13	**Save buttons**	Save all three plots as one PNG or SVG image, or the report as a text file.
14	**View**	The selected plot or report. The plot fills the free height of the window and keeps a 4:3 plot area. Drag across a plot to choose the fit range; double-click it to use all data (Section 5.4).
15	**Plot hint**	Explains range selection; shows the selected range while you drag.
16	**Axis scales**	Set the temperature axis (0–100 °C) and/or the period axis (0–180 h) manually; unticked axes are automatic (Section 5.5).

5.3 Step by step

1. **Look at the demo.** The page opens with the example data already calculated, so you can see what the results look like.
2. **Clear the form.** Click **Clear**. The notice, the example data and the results disappear.
3. **Enter your data.** Type or paste pairs into the data box, or use **Load CSV…**. Optionally enter a plot title.
4. **Choose the base temperature.**

5. **Calculate.** Click **Calculate** or press Ctrl+Enter. The summary row and the plot update. If something is wrong with the input, a red message below the button explains what (Section 11).
6. **Refine.** From now on most changes take effect at once, without clicking **Calculate** again: the base temperature, the fit range (typed into Low and High, or selected on a plot, Section 5.4), the axis scales (Section 5.5), the polynomial fit (Section 5.6), the plot title and the Arrhenius-line and tab-delimited options. Only a change to the data needs **Calculate**.
7. **Review.** Switch between the tabs.
8. **Save.** Use the save buttons (Section 5.9).

5.4 Setting the fit range

The Q_{10} fit can be restricted to a temperature range in two ways. Both update all results at once and keep the current tab.

Typing the range

Tick **Use only data within a temperature range** and enter Low and High. After a first calculation, the results update as you type: the calculation waits until you pause (about half a second), so an intermediate value such as “3” on the way to “38” is not used; Enter or moving to another field applies it immediately. An empty field is treated as unfinished. **Use all data** switches the range off.

Selecting the range on a plot

1. **Drag** horizontally across the plot area of any of the three plots. A band shows the selection, and the hint below the plot shows its temperatures, for example “Selected: 28.0–40.0 °C — release to recalculate”.
2. **Release.** The range is written into Low and High, the range box is ticked, and the fit is recalculated. The summary row shows how many points are used (for example “13 of 18”).
3. **To go back to all data,** double-click a plot or click **Use all data**.

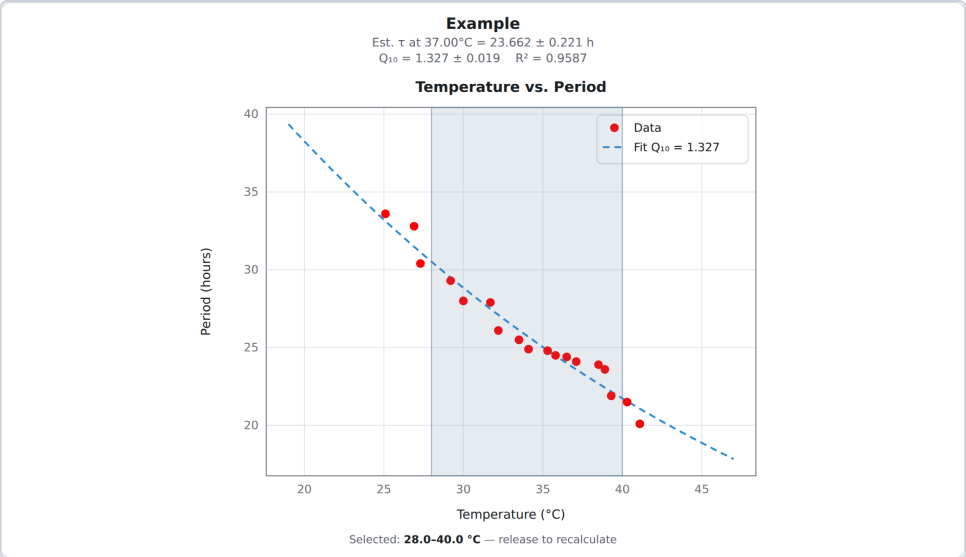


Figure 2. Selecting 28–40 °C by dragging across the Period plot. The hint below the plot shows the selected range.

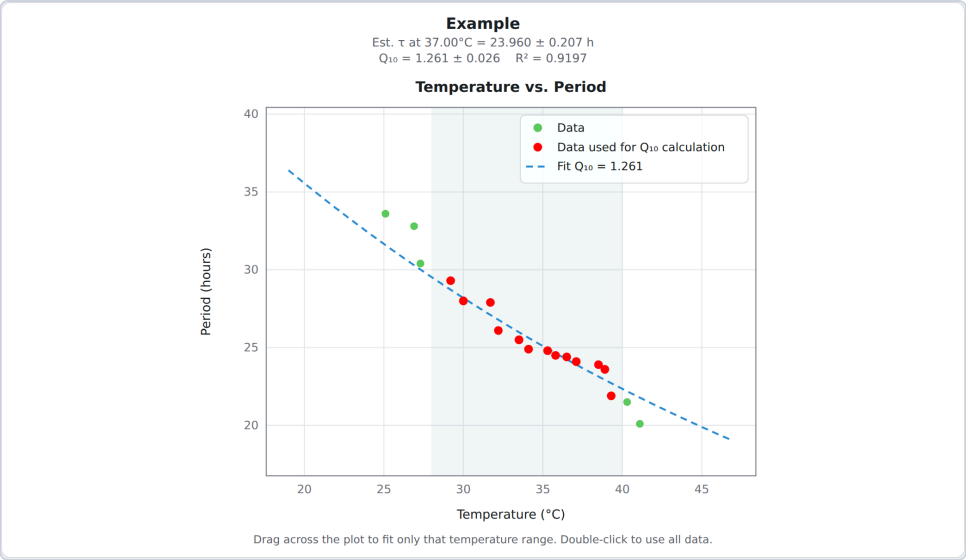


Figure 3. The same plot after release. The fit range is shaded, points outside it are light green, and the axis scales are unchanged.

- **Any plot works.** On the Arrhenius plot the horizontal axis is 1000/T; the selection is converted back to temperature automatically.
- **Rounding.** A range selected on a plot is rounded outwards to 0.1 °C (Low down, High up), so points at the edges of the band are included. Because a mouse position is resolved only to one screen pixel, the result can be 0.1 °C wider than intended; type exact values into Low and High if you need them.
- **Too few points.** If a range contains fewer than 3 points, a red message appears below **Calculate** and the previous results are kept (after a selection on a plot, the previous range is also restored).
- **Clicks are ignored.** A click or a very short drag does not change anything.
- **Touch screens.** On phones and tablets, swipe horizontally across the plot; vertical swipes still scroll the page.
- **Fixed scales.** The axis scales always match the all-data fit (Section 3.6), so plots for different ranges can be compared side by side.
- **Saved figures** do not include the shaded band; points outside the range are shown in light green as usual.

5.5 Setting the axis scales

By default the axis scales are calculated automatically (Section 3.6). To set them yourself, use the **Axis scales** row below the plot:

1. Tick **Temperature** and/or **Period**. The first time, Min and Max are filled in with the current automatic limits of the Period plot, rounded outwards to 0.5 (for the example data 17.5–48.5 °C and 16.5–40.5 h), so you can adjust them from there. The boxes become available after a first calculation.
2. Enter Min and Max: temperature between 0 and 100 °C, period between 0 and 180 h; the arrows of the fields change the values in steps of 0.5. The plots are redrawn as soon as a value is confirmed (Enter, or moving to another field); nothing is recalculated.
3. Untick a box to return that axis to automatic scaling.

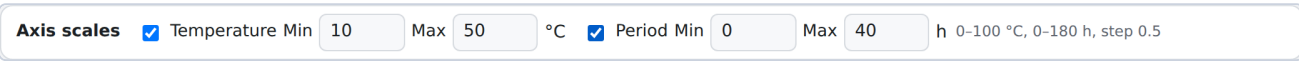


Figure 4. The Axis scales row with a temperature axis of 10–50 °C and a period axis of 0–40 h.

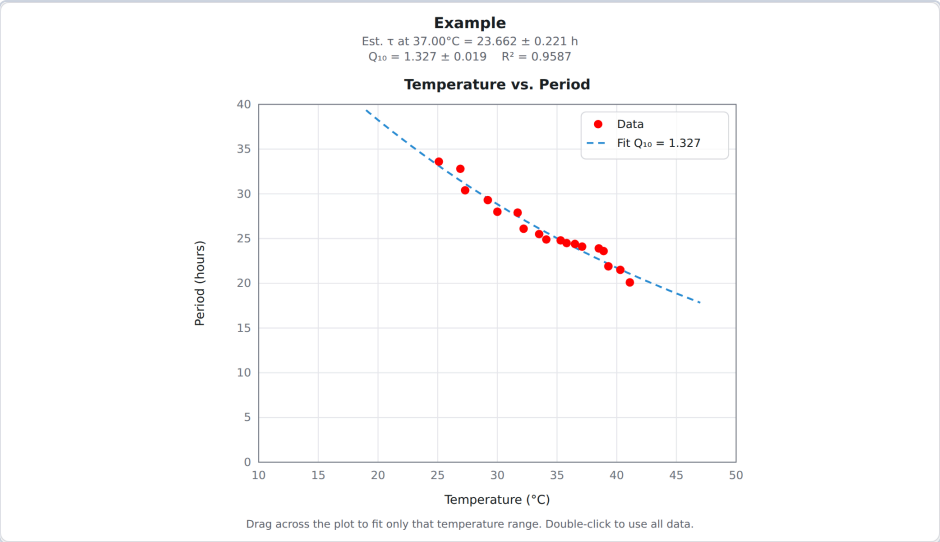


Figure 5. The Period plot with these manual scales.

- **All three plots** use the ranges, converted as described in Section 3.6; for example, the Arrhenius x-axis becomes 1000/(50 + 273.15) to 1000/(10 + 273.15), i.e. 3.0945 to 3.5317 K^{-1}.
- **Saved figures** use the same scales as the screen.
- **The settings are kept** when you recalculate, select a fit range, or click **Clear**.
- **Invalid values** — a missing value, a value outside the allowed range, or Min not smaller than Max — are reported in red below **Calculate**, and that axis is scaled automatically until the values are corrected.
- **Comparing datasets.** Manual scales are useful for drawing several datasets (for example different strains) with identical axes, so that the saved figures can be compared or combined directly.

5.6 Polynomial fit (optional)

In addition to the Q_{10} model, a polynomial of period on temperature can be fitted, for example to describe the shape of the temperature dependence without assuming a model. It is off by default and does not affect the Q_{10} fit or the Arrhenius regression.

1. Tick **Fit a polynomial** and choose the **Degree** (1–6; default 2).
2. The polynomial uses **all data**, or — when **Range** is ticked — only the data between its own Low and High. This range is independent of the Q_{10} fit range.
3. Changes take effect at once (after a first calculation).

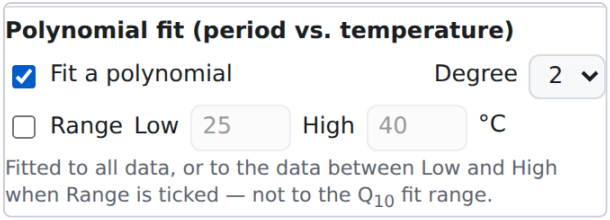


Figure 6. The Polynomial fit options.

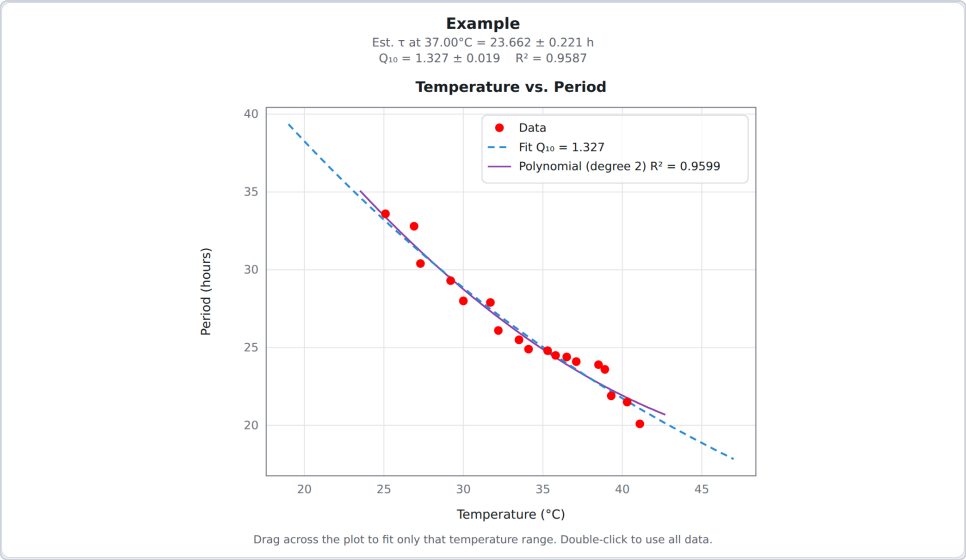


Figure 7. A polynomial of degree 2 fitted to all data, shown as a purple line on the Period plot.

- **The line** is drawn on the Period plot over the temperature span of *all* data, extended by 10 % of that span on each side (for the example data 23.5–42.7 °C), like the Q_{10} curve. When a range is used, the parts outside it are extrapolations of the polynomial; polynomials, especially of high degree, can behave badly outside the data they were fitted to.
- **The report** always includes, when the option is on: the degree, the number and temperature range of the points used, the equation, the coefficients a_0 … a_d with standard errors, R^2, adjusted R^2 and the residual standard error (Section 8.1). The 200 calculated points are added only when **Also show tab-delimited data in the report** is ticked (Section 8.3).
- **Too few points.** A polynomial of degree d needs at least d + 2 points (d + 1 coefficients plus one degree of freedom for the standard errors). Otherwise, or if Low is not smaller than High, a red message appears and no polynomial is drawn.
- **Saved figures** include the line; the axis limits of the Period plot are widened if needed to show it.

The calculation is described in Section 3.9.

5.7 The plots

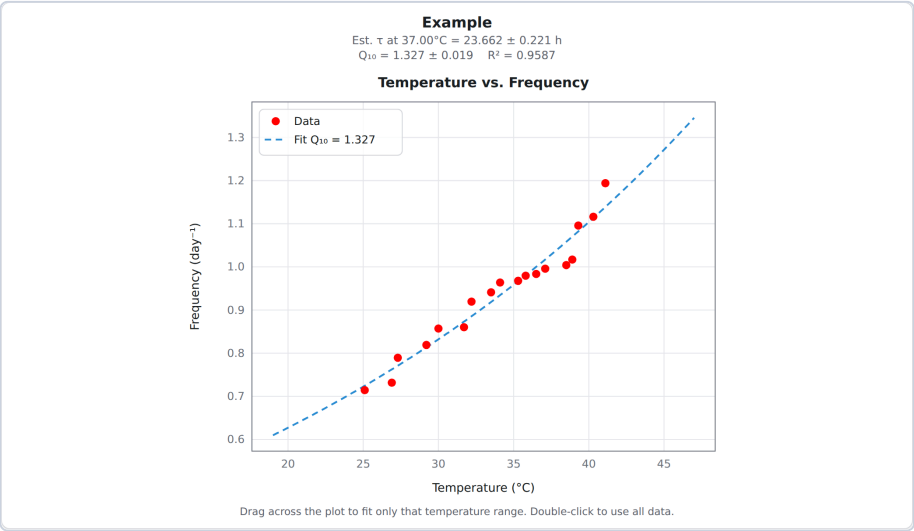


Figure 8. Frequency tab: frequency (day^{-1}) against temperature with the fitted Q_{10} curve.

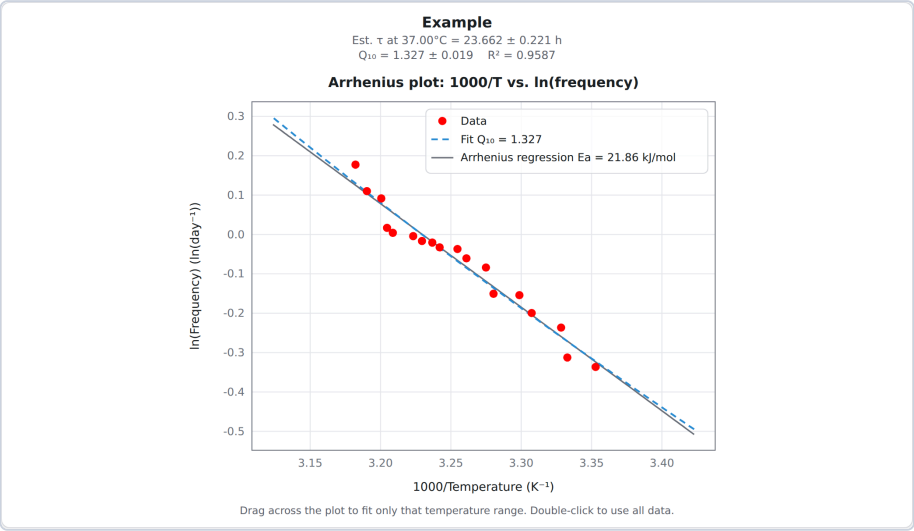


Figure 9. Arrhenius tab: ln(frequency) against 1000/T, with the fitted Q_{10} curve (dashed) and the Arrhenius regression line (solid grey).

Each plot shows the title and a two-line summary (τ_b, Q_{10} and R^2) above it, and its plot area always has a width-to-height ratio of 4:3. Lines are drawn below the data points, so they never hide a point. The legend is placed automatically in the corner where it covers the fewest points. When a fit range is set, it is shaded and the legend distinguishes the points used from those outside the range (Figure 3).

5.8 The report

The **Report** tab shows the complete text results: the input table with summary statistics, the data used for the fit (when a range is set), optional tab-delimited data, the Q_{10} and Arrhenius results, and the polynomial fit (when switched on). Its contents are described in Section 8; the text is identical to the file saved with **Save report (TXT)**.

5.9 Saving

Button	What is saved
Save figure (PNG)	All three plots stacked vertically under the title and summary, as a high-resolution (2×) PNG image with a white background.
Save figure (SVG)	The same figure as a vector SVG file, which can be edited in Illustrator, Inkscape or similar programs and scales without loss.
Save report (TXT)	The report as plain UTF-8 text.

File names are built from the plot title, for example `Example_q10_plots.png` and `Example_q10_results.txt` (or `q10_…` when there is no title). Files go to your browser's usual download location. The saved figure reflects the current Arrhenius-line option, axis scales and polynomial fit.

5.10 The "How it works" window

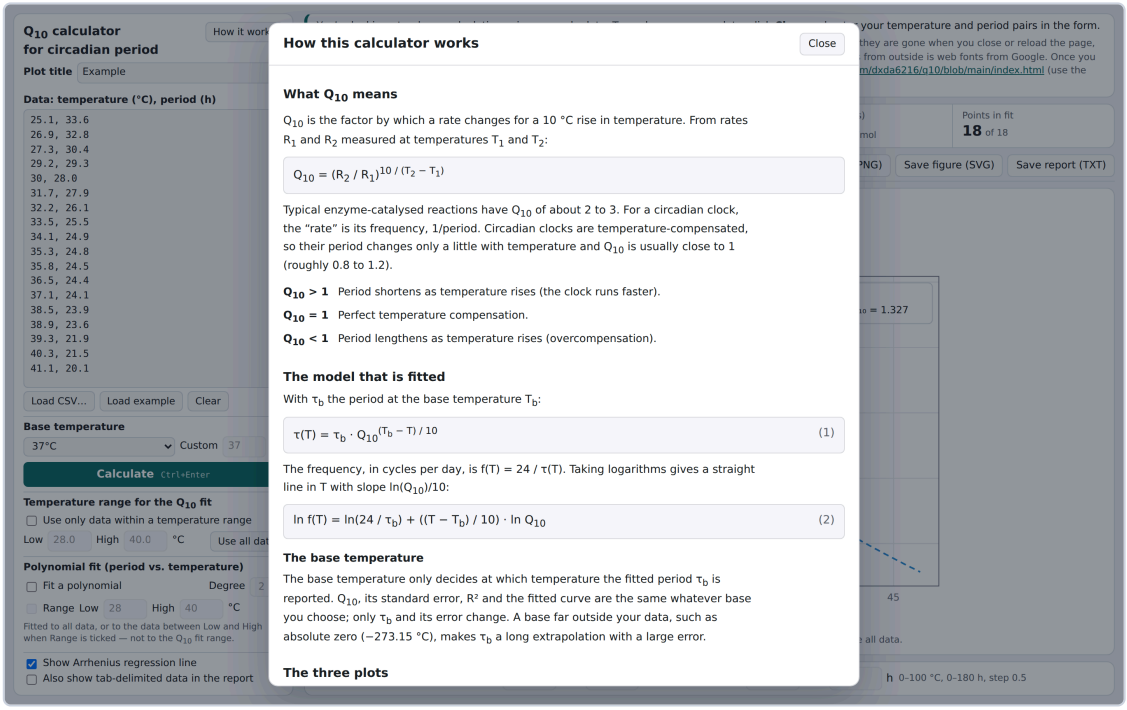


Figure 10. The "How it works" window, opened with the button at the top of the input panel. It closes with Close, the Esc key, or a click outside it.

The window summarises the theory, the calculation, usage and privacy (Sections 2, 3 and 12 of this manual in shorter form) and contains the download link for the HTML file.

5.11 Phones, tablets and dark mode

On narrow screens the page scrolls: the input panel is placed above the results, the summary row wraps into two columns and plots are drawn at a size that keeps their text legible.

The page also follows the device's light or dark appearance setting. Saved figures always use a white background, whichever mode is active.

Opening a downloaded HTML file on a phone may require a file manager app or the browser's file-open function; on some devices it is easier to use the online web app.

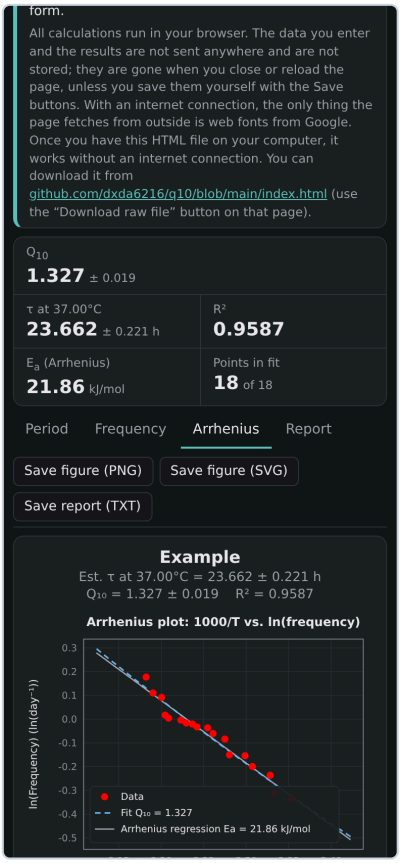


Figure 11. The browser version on a phone in dark mode.

6 Python version

6.1 Requirements

Component	Requirement
Python	3.9 or later
numpy	Any current version
SciPy	Any current version (provides `curve_fit`)
matplotlib	3.3 or later (needed for the layout of the saved three-plot figure)
Tkinter	Part of the Python standard library; usually included (see below)

All other modules used (`csv`, `math`, `re`, `statistics`, `webbrowser`) are part of the standard library. pandas is not needed.

6.2 Installation

Install the three packages with pip:

```
pip install numpy scipy matplotlib
```

Whether Tkinter is already present depends on how Python was installed:

Python installation	Tkinter
Windows or macOS, installer from python.org	Included
macOS with Homebrew Python	`brew install python-tk`
Debian, Ubuntu	`sudo apt install python3-tk`
Fedora	`sudo dnf install python3-tkinter`
Anaconda / Miniconda	Included (see Section 6.3)

To check that Tkinter works, run `python -m tkinter`; a small test window should appear.

6.3 Anaconda and Miniconda

The full Anaconda distribution already contains Python, numpy, SciPy, matplotlib and Tkinter, so nothing needs to be installed. With Miniconda, install the three packages with conda:

```
conda install numpy scipy matplotlib
```

To keep the calculator separate from other projects, you can create a dedicated environment:

```
conda create -n q10 python=3.12 numpy scipy matplotlib
conda activate q10
```

Avoid mixing `pip install` and `conda install` in the same environment, as this can break dependencies.

> **Spyder:** running the program inside Spyder can make its window freeze or fail to appear, because the Tkinter window conflicts with Spyder's own matplotlib handling. Run it from the Anaconda Prompt or a terminal instead, or set **Run → Configuration per file → Execute in an external system terminal**.
>
> **Jupyter:** the .py file is a desktop window application and is not meant to run inside a notebook; use the notebook version instead (Section 7.5).
>
> **Linux:** with older conda builds of Tk, text in the window can look jagged. This is cosmetic; `conda install -c conda-forge tk` may improve it.

6.4 Starting the program

Open a terminal (on Windows with Anaconda: the Anaconda Prompt), change to the folder containing the file and run:

```
python q10_calculator.py
```

On some systems the command is `python3`. The window opens with the example data already calculated and a notice above the results.

6.5 The window

The window opens at 1280 pixels wide and fits a screen 900 pixels high; it can be made larger. Its layout follows the browser version (Figure 1):

- **Left panel**, from top to bottom: the program name with **How it works**; **Plot title** on one line; the data box with **Load CSV…**, **Load example** and **Clear**, which takes all the remaining height; **Base temperature** with the Custom field; **Calculate (Ctrl+Enter)**, with a message line below it that appears only for problems (in red) and while you drag on a plot; **Temperature range for the Q10 fit** with **Use all data**; **Polynomial fit (period vs. temperature)**; the Arrhenius-line and tab-delimited options; and **Save figure…** and **Save results…**.
- **Right side:** the start-up notice (demo, privacy statement and the download link for the browser version); four tabs — **Results** (the text report) and the **Period**, **Frequency** and **Arrhenius** plots; and the **Axis scales** row below the tabs.

Each plot keeps a 4:3 plot area directly under its title and summary, and has the standard matplotlib toolbar below it for zooming, panning and saving that single plot. After **Calculate** the **Period** tab is selected.

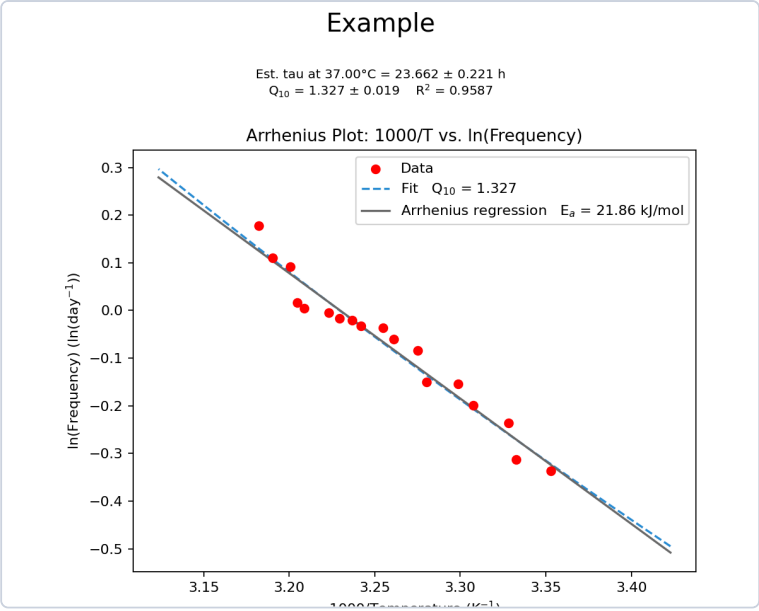


Figure 12. The Arrhenius tab of the Python version with the example data.

6.6 Step by step

The procedure is the same as for the browser version (Section 5.3): click **Clear**, enter or load your data, choose the base temperature and click **Calculate** or press Ctrl+Enter. If the input cannot be used, a dialog box titled “Cannot calculate” explains why, and the message is also shown in red below the button. After a first calculation, changes to the base temperature, the fit range, the axis scales, the polynomial fit, the plot title and the options take effect at once.

Setting the fit range

As in the browser version (Section 5.4), you can type Low and High — the results update when you pause, press Enter or leave the field, and problems are shown in red below **Calculate** without a dialog — or drag horizontally across any plot. While you drag, a band marks the selection and the line below **Calculate** shows its temperatures. On release, the range is entered into Low and High, the range box is ticked and the fit is recalculated; the current tab stays selected. The range is shaded on the plots, and the axis scales stay those of the all-data fit. Double-click a plot or click **Use all data** to return to all data. Rounding to 0.1 °C, the handling of ranges with fewer than 3 points and the omission of the band from saved figures are the same as in the browser version.

> **Zoom and pan:** while the toolbar’s zoom or pan mode is switched on, dragging zooms or pans instead, and double-clicking does not reset the range. Click the active toolbar button again to switch the mode off. The toolbar’s Home button returns to the fixed (or manual) scales.

Setting the axis scales

The **Axis scales** row below the tabs has **Temperature** (0–100 °C) and **Period** (0–180 h), each with a tick box and Min and Max fields (arrows in steps of 0.5). It works as in the browser version (Section 5.5): ticking an axis for the first time fills in the current automatic limits; a change is applied when you press Enter, leave the field or use the arrows; invalid values are reported in red below **Calculate** and that axis stays automatic; saved figures use the same scales.

Polynomial fit

The **Polynomial fit** box works as in the browser version (Section 5.6): tick **Fit a polynomial**, choose the degree and optionally its own range. The purple line appears on the Period tab, and Results gains the polynomial section; the 200 calculated points are included when **Also show tab-delimited data in Results** is ticked, and switching that option rewrites Results at once.

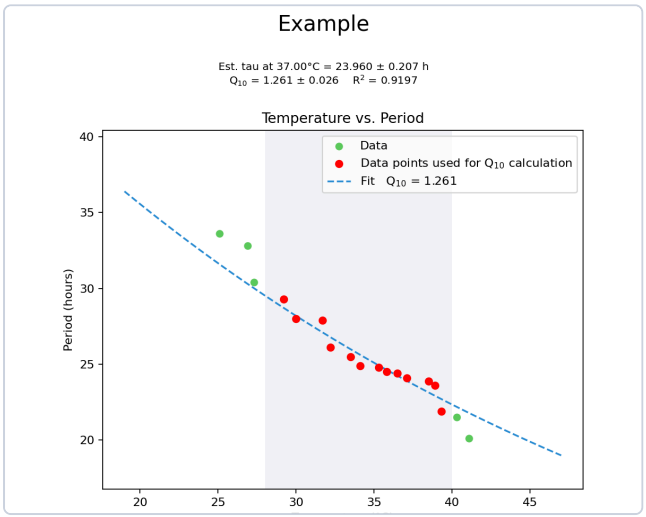


Figure 13. The Period tab of the Python version after selecting 28–40 °C on the plot.

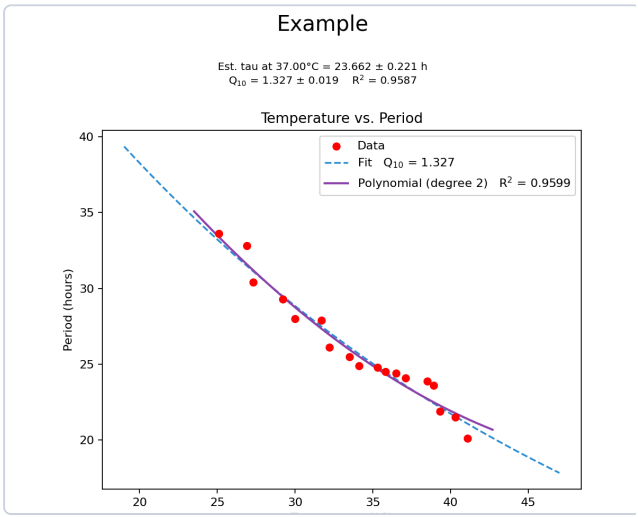


Figure 14. The Period tab of the Python version with a polynomial of degree 2.

6.7 Saving

Save figure… asks for a file name and saves the three plots stacked in one image, in PNG (150 dpi), PDF or SVG format according to the extension you choose. PDF and SVG are vector formats suitable for publication figures. The figure includes the Arrhenius line, the manual axis scales and the polynomial as currently set.

Save results… saves the Results text as a UTF-8 text file.

A single plot can also be saved from its tab with the floppy-disk button on the matplotlib toolbar.

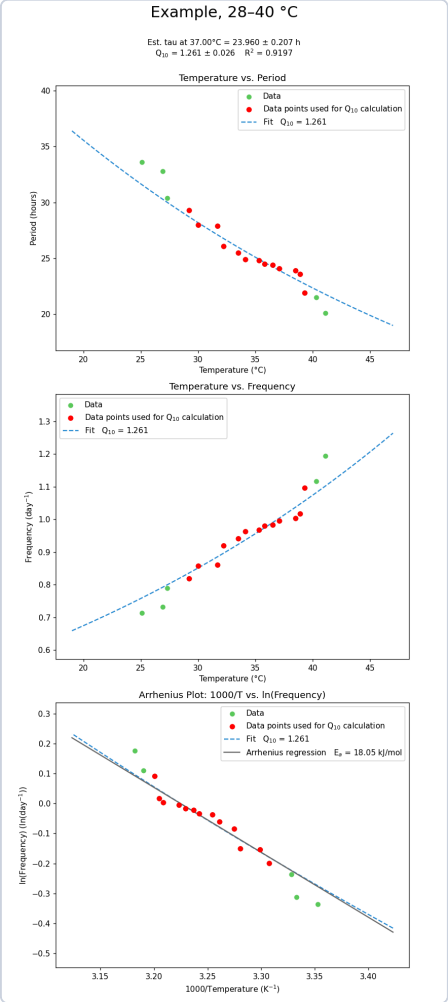


Figure 15. A figure saved by the Python version (fit range 28–40 °C).

7 Google Colab version

7.1 Opening the notebook

A Google account is required. The quickest way is to open colab.research.google.com/github/dxda6216/q10/blob/main/circadian_period_q10.ipynb, which loads the notebook from the repository into Colab. Alternatively, in Google Colab (colab.research.google.com), choose **File → Open notebook**, select the **GitHub** tab, enter `dxda6216/q10` and select the notebook. Alternatively, download `circadian_period_q10.ipynb` and use **File → Upload notebook**.

The notebook has three parts: a short introduction with a privacy note, the calculator cell (shown as a form), and a "How this calculator works" section with the theory and instructions. When it opens, the form contains the example data and the output below it already shows the corresponding result.

7.2 Form fields

Field	Description
`Data_description`	Optional plot title.
`Temperatures`	Temperatures in °C, separated by commas.
`Periods`	Periods in hours, separated by commas, in the same order as the temperatures.
`Set_temperature_range`	"Yes" to fit only the points within the range set by the two sliders; "No" to use all points.
`Range_low_limit`, `Range_high_limit`	Range limits in °C (sliders, 0–65 °C, step 0.1). Used only when the range is switched on. The axis scales of the plots stay those of the all-data fit, so plots for different ranges can be compared directly.
`Select_base_temperature`	Base temperature option (Section 4.5). Default 30 °C.
`Base_temperature`	Custom base temperature (slider, 0–100 °C), used when "Set "Base_temperature" by slider below" is selected.
`Temperature_axis`	"Auto" (default) for automatic scaling, or "Manual" to use the two sliders below.
`Temperature_axis_min`, `Temperature_axis_max`	Temperature axis range in °C (sliders, 0–100 °C, step 0.1). Used only when Temperature_axis is "Manual".
`Period_axis`	"Auto" (default) or "Manual", as for the temperature axis.
`Period_axis_min`, `Period_axis_max`	Period axis range in hours (sliders, 0–180 h, step 0.1). Used only when Period_axis is "Manual". The frequency and Arrhenius axes are converted from the ranges (Section 3.6). If Min is not smaller than Max, a "Note:" line is printed and that axis is scaled automatically.
`Show_Arrhenius_regression_line`	"Yes" (default) to draw the Arrhenius regression line and its E_a on the Arrhenius plot.
`Display_tab_delimited_data`	"Yes" to print the dataset, fitted curve and regression line as tab-separated columns.

7.3 Running and output

After editing the form, run the cell with **Runtime → Run all**, Ctrl + F9, or the cell's play button. Changes to the form take effect only when the cell is run again. The first run of a session may take a few seconds while Colab starts a runtime.

The output contains, in order: the demo notice (only while the form still holds the example data), the input table and statistics, the data used for the fit (when a range is set), the tab-delimited data (if chosen), the RESULTS and ARRHENIUS REGRESSION blocks, and the three plots.

If the input cannot be used, the output shows a single line starting with "Cannot calculate:" instead (Section 11).

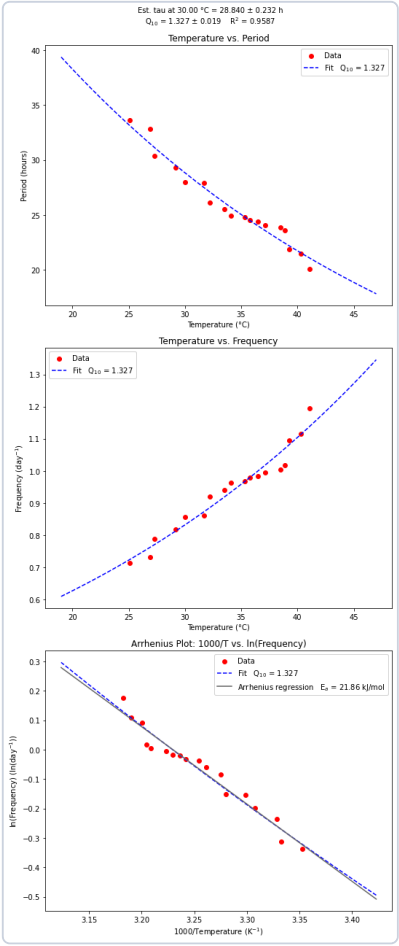


Figure 16. The plots produced by the Colab version with the example data (base temperature 30 °C).

7.4 Saving

To save the figure, right-click it and choose **Save image as…** (PNG). To keep the results, select and copy the printed text; set `Display_tab_delimited_data` to "Yes" to get columns that paste directly into a spreadsheet. Saving the notebook itself (for example with **File → Save a copy in Drive**) also keeps the form values and the output. Notebooks are always saved to your own account or computer, never to the author's; see Section 12 before sharing one.

7.5 Using the notebook in Jupyter

The notebook also runs in Jupyter Notebook or JupyterLab (it additionally needs pandas). The form fields are a Colab feature, so in Jupyter the code is shown in full; edit the values in the "inputs & run" section near the end of the cell (for example `Temperatures = …`) and run the cell.

8 Output reference

8.1 Results text

The text results consist of the following blocks (the Colab version prints the input tables with pandas, so their layout differs slightly):

Block	Contents
INPUT DATA	Numbered table of all temperatures and periods, followed by the minimum, maximum, average and median of each.
DATA USED FOR Q10 CALCULATION	The same for the points inside the fit range. Only shown when a range is set.
DATASET, FITTED CURVE, ARRHENIUS REGRESSION LINE	Tab-delimited columns (Section 8.3). Only shown when chosen.
RESULTS	Estimated period at the base temperature ± SE (hours); Q_{10} ± SE; R^2 of the Q_{10} fit.
ARRHENIUS REGRESSION	E_a ± SE (kJ/mol); slope d ln(f)/d(1000/T) ± SE (in units of 10^3 K); intercept ± SE; R^2 of the regression.
POLYNOMIAL FIT *(browser, Python; when switched on)*	Degree; number and temperature range of the points used; the equation; coefficients a_0 … a_d ± SE; R^2, adjusted R^2 and residual standard error. Example (degree 2, all data): `Period = 70.43496 - 1.919776*T + 0.01767489*T^2`.

Example of the final two blocks (example data, base temperature 37 °C):

```
RESULTS
Estimated period length at 37.00 °C = 23.662 ± 0.221 hours
Q10 (temperature coefficient) = 1.327 ± 0.019
R² = 0.958656

ARRHENIUS REGRESSION (ln(Frequency) vs. 1000/T, data used for Q10 calculation)
Activation energy Ea = 21.859 ± 1.178 kJ/mol
Slope d ln(f) / d(1000/T) = -2.6290 ± 0.1416 (10³ K)
Intercept = 8.4911 ± 0.4612
R² = 0.955620
```

8.2 Plots

Plot	x-axis	y-axis
Temperature vs. Period	Temperature (°C)	Period (hours)
Temperature vs. Frequency	Temperature (°C)	Frequency = 24/period (day^{-1})
Arrhenius plot	1000/Temperature (K^{-1})	ln(Frequency) (ln(day^{-1}))

Element	Meaning
Red points	Data used in the fit.

Element	Meaning
Light-green points	Data outside the chosen temperature range (plotted, not used).
Dashed blue line	The fitted Q_{10} model, labelled with its Q_{10}.
Solid grey line (Arrhenius plot only)	The Arrhenius regression line, labelled with its E_a.
Solid purple line (Period plot only; browser and Python)	The optional polynomial fit, labelled with its degree and R^2.
Grey-blue band (browser and Python, on screen only)	The current fit range.

The axis scales are the same with and without a fit range (Section 3.6), unless they are set manually. Points and curves that leave the plot area are clipped at the frame.

In the Arrhenius plot, higher temperatures are on the *left* (smaller 1000/T) and higher frequencies at the top, so a clock that speeds up with warming gives a line falling from left to right.

8.3 Tab-delimited data

With the tab-delimited option, three tables are added, each with a header row and tab-separated columns that can be pasted directly into Excel, Google Sheets or a graphing program to redraw the plots:

- **Dataset:** Temp (°C), Period (hours) for all points;
- **Fitted curve:** Temp (°C), Period (hours) at the 200 points of the fitted curve;
- **Arrhenius regression line:** Temp (°C), 1000/T (1/K), ln(Frequency) at the same 200 temperatures;
- **Polynomial fit curve** (browser and Python, when the polynomial fit is on): Temp (°C), Period (hours) at 200 temperatures spanning all data plus 10 % of the span on each side.

9 Worked example

All three versions open with the following 18 data points. Running the calculator on them is a quick way to check that your copy works: you should obtain exactly the values below.

#	Temperature (°C)	Period (h)	#	Temperature (°C)	Period (h)
0	25.1	33.6	9	35.3	24.8
1	26.9	32.8	10	35.8	24.5
2	27.3	30.4	11	36.5	24.4
3	29.2	29.3	12	37.1	24.1
4	30	28.0	13	38.5	23.9
5	31.7	27.9	14	38.9	23.6
6	32.2	26.1	15	39.3	21.9
7	33.5	25.5	16	40.3	21.5
8	34.1	24.9	17	41.1	20.1

9.1 All points, base temperature 37 °C

Quantity	Value
Estimated period at 37.00 °C	23.662 ± 0.221 h
Q_{10}	1.327 ± 0.019
R^2 (Q_{10} fit)	0.958656
E_a (Arrhenius regression)	21.859 ± 1.178 kJ/mol
Slope	−2.6290 ± 0.1416 (10^3 K)
Intercept	8.4911 ± 0.4612
R^2 (Arrhenius regression)	0.955620

With a base temperature of 30 °C (the Colab default) the estimated period becomes 28.840 ± 0.232 h; all other values are unchanged.

9.2 Fit range 28–40 °C

With the range option on (Low 28, High 40), 13 of the 18 points are used. The results are Q_{10} = 1.261 ± 0.026, R^2 = 0.9197 and E_a = 18.051 ± 1.646 kJ/mol.

9.3 Polynomial fit (browser and Python)

With **Fit a polynomial** on, degree 2 and all data:

```
POLYNOMIAL FIT (Period vs. Temperature, degree 2)
Data used: 18 points, 25.100–41.100 °C (T in °C, Period in hours)
Period = 70.43496 - 1.919776*T + 0.01767489*T^2
Coefficients (± SE):
  a0 (T^0) = 70.43496 ± 9.858902
  a1 (T^1) = -1.919776 ± 0.5999799
  a2 (T^2) = 0.01767489 ± 0.008985137
R² = 0.959873   adjusted R² = 0.954522   residual SE = 0.7932 h
```

The line is drawn from 23.5 to 42.7 °C (the data range 25.1–41.1 °C plus 10 % of its width on each side).

9.4 Reading the example

A Q_{10} of 1.33 means that in this (illustrative) dataset the clock's frequency rises by about a third for every 10 °C of warming — the period shortens from about 33.6 h at 25 °C to about 20 h at 41 °C. That is markedly less temperature-sensitive than a typical enzyme reaction ($Q_{10} \approx 2$–3), but less well compensated than many circadian clocks ($Q_{10} \approx 0.8$–1.2). Converting with equation (4) over the data range (25.1–41.1 °C) gives $E_a \approx 22.0$ kJ/mol, in close agreement with the 21.9 kJ/mol from the Arrhenius regression. Restricting the fit to 28–40 °C lowers Q_{10} to 1.26, showing that the steepest change in period lies near the ends of the range.

10 Interpreting the results

Q_{10} and its standard error

The standard error describes the statistical uncertainty of Q_{10} given the scatter of your data about the fitted curve. An approximate 95 % confidence interval is $Q_{10} \pm t \cdot SE$, where t is the 97.5 % point of Student's t distribution with n − 2 degrees of freedom: 2.57 for n = 7, 2.31 for n = 10, 2.16 for n = 15 and 2.12 for n = 18. For larger datasets t approaches 2. The standard error assumes that the residuals are independent and of similar size at all temperatures; it does not cover systematic errors such as inaccurate temperature control.

Testing temperature compensation

A Q_{10} whose confidence interval includes 1 is consistent with perfect compensation over the temperature range studied. To compare two strains or conditions, compare their Q_{10} values together with their standard errors, fitted over the same temperature range.

R^2

R^2 measures how much of the variation in period the model explains, not how well the clock is compensated. In a well-compensated clock the period barely changes, so there is little variation to explain, and R^2 can be low even when the model is adequate. Judge the fit mainly by looking at the plots: the points should scatter randomly around the curve, without a systematic bend.

Q_{10} versus E_a

Q_{10} and E_a are two descriptions of the same temperature dependence (Section 2.5). Q_{10} is the common measure in circadian biology; E_a allows comparison with biochemical reactions. If the two lines in the Arrhenius plot clearly diverge *within* the data, or the points curve away from both, neither simple model describes the data well over the whole range; consider restricting the range.

Comparing ranges

Because the axis scales do not change with the fit range, you can try several ranges and compare the plots and Q_{10} values directly. This is a quick way to see whether the temperature dependence is uniform: if Q_{10} changes markedly between ranges, as in the example (1.33 for all data, 1.45 for 25–30 °C, 1.29 for 33–41.1 °C), report the range you used and consider describing the ranges separately. Avoid choosing a range only because it gives the result you expected.

Extrapolation

The fitted curve and the regression line are drawn beyond the data for orientation only. Estimates of period far outside the measured temperatures — including τ_b at a distant base temperature — are extrapolations and should be treated with caution.

What to report

Report $Q_{10} \pm$ SE, the number of points and the temperature range used, whether the points were individual measurements or means, and — if you give a period estimate — the base temperature.

11 Error messages and troubleshooting

11.1 Error messages

When the input cannot be used, no results are produced and a message explains why. The browser version shows it in red directly below **Calculate** (this line is shown only when there is a problem), the Python version in a dialog box and in red below **Calculate** (for live changes such as typing a range, only below the button), and the Colab version as a line starting with “Cannot calculate:”. The previous results are kept until the problem is corrected; when the error follows a range selected on a plot, the previous range is also restored.

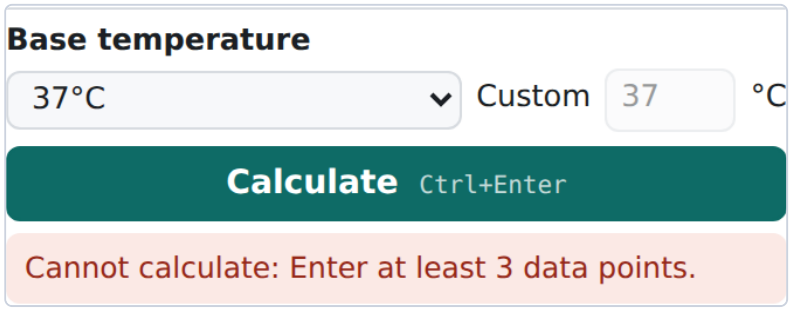


Figure 17. An error message below Calculate in the browser version.

Message	Cause and remedy
Number of temperatures (*n*) and periods (*m*) differ.	A value is missing or extra. Give exactly one period per temperature.
Enter at least 3 data points.	Fewer than 3 pairs were entered. Add data.
All periods must be positive.	A period is zero or negative. Check for typos.
Range low limit must be smaller than the high limit.	Swap or correct the range limits.
Only *n* data point(s) in the selected range; need at least 3.	The range is too narrow for your data. Widen it.
Line *n*: cannot read numbers: “…” *(browser, Python)*	That line contains text that is not a number (other than one header line). Correct or delete it, or start it with # to make it a comment.
Line *n*: expected 2 values (temperature, period), found *k* *(browser, Python)*	A line has too many or too few values. Each line must hold exactly one pair.
Temperatures: could not read the numbers. Separate them with commas. *(Colab)*	The list contains something other than numbers and commas.
Enter a custom base temperature. *(browser)*	“Custom value” is selected but the field is empty.
The fit did not converge for these data. *(browser)* / Optimal parameters not found… *(Python, Colab)*	Very rarely, unusual data prevent the fit from finding a solution. Check the data for errors and outliers.
Temperature axis: enter Min and Max. / Period axis: … *(browser, Python)*	A manual axis is ticked but a field is empty. That axis is scaled automatically until both values are entered.

Message	Cause and remedy
Temperature axis: values must be between 0 and 100 °C. / Period axis: … between 0 and 180 h.	Enter values inside the allowed range. That axis is scaled automatically meanwhile.
Temperature axis: Min must be smaller than Max. / Period axis: … *(Colab: printed as a "Note:" line)*	Swap or correct the two values. That axis is scaled automatically meanwhile.
Polynomial of degree *d* needs at least *d* + 2 data points (*n* in its range). *(browser, Python)*	Choose a lower degree or widen the polynomial's range. The Q_{10} results are not affected.
Polynomial fit: range Low must be smaller than High. *(browser, Python)*	Correct the polynomial's Low and High.
Load failed: no numeric rows found (need two numeric columns). *(browser, Python)*	The CSV file has no row whose first two cells are both numbers. Check the column order and the separator.

11.2 Troubleshooting

Problem	Solution
Browser: a save button seems to do nothing	Check the browser's download list or permissions; some browsers ask before saving several files from one page. On iOS, files may open in a new tab, from which they can be shared or saved.
Browser: the fonts look different	Without an internet connection the page uses your system fonts. This is expected and does not affect the results.
Python: `ModuleNotFoundError: No module named 'numpy'` (or scipy, matplotlib)	Install the packages (Section 6.2) in the same Python environment that runs the program.
Python: `No module named 'tkinter'` or `_tkinter`	Install Tkinter for your system (Section 6.2).
Python: the window freezes or does not appear in Spyder	Run the program from a terminal (Section 6.3).
Python: text looks jagged on Linux with conda	Cosmetic; see Section 6.3.
Python: dragging on a plot zooms or pans instead of selecting a range	The toolbar's zoom or pan mode is on. Click the highlighted toolbar button to switch it off.
Browser, Python: the selected range is 0.1 °C wider than intended	Expected (rounding outwards to 0.1 °C, Section 5.4). Type exact values into Low and High.
Colab: changes in the form have no effect	Run the cell again after editing the form.
Colab: "Runtime disconnected"	Colab stops idle runtimes. Run the cell again; the form values are kept.

12 Privacy and data handling

Browser and Python versions

- **Calculations run on your own computer.** The fit, the plots and the report are computed locally.
- **Nothing is sent.** The data you enter and the results are never transmitted. The browser version contains no code that uploads, transmits or reports anything; the Python version makes no network connections.
- **Nothing is stored.** Data and results exist only while the page or program is open. The browser version uses no cookies or browser storage and asks the browser not to remember form contents. Files are written only when you click a Save button, and only to the place you choose.
- **Network access.** With an internet connection, the browser version downloads its typefaces from Google Fonts. That request contains none of your data, but, like any web request, lets Google see your IP address and browser type. Offline, system fonts are used and everything else works unchanged. The Python version works fully offline; matplotlib may keep its own font cache, which contains no data you enter.
- **The online web app.** When you open the browser version at dxda6216.github.io, the page itself is downloaded from GitHub Pages, so GitHub receives the usual information of a web request, such as your IP address. Your data and results are still processed only in your browser and are not sent to GitHub or anywhere else.
- **The GitHub link.** Both versions show a link to the download page of the browser version. It is opened only if you click it.

Colab version

- **The calculation runs under your own Google account.** When you open the notebook, Colab runs it in a runtime connected to the Google account you are signed in with — not the account of the calculator's author. The runtime is on Google's servers, so your data is processed by Google Colab under Google's terms. The author of the calculator has no access to your runtime, your data or your results.
- **Opening the notebook from GitHub does not change the repository.** Colab loads a copy of the notebook from the repository. Your edits and results are never written back to the repository, and until you save the notebook they exist only in your Colab session.
- **Anything you save is saved to your own account.** Form values are part of the notebook's code, and the printed results and plots are kept in its output, so they are included whenever the notebook is saved:
  - **File → Save a copy in Drive** saves it to *your* Google Drive.
  - **File → Upload notebook** also stores the uploaded notebook in *your* Google Drive.
  - **File → Save a copy in GitHub** saves it to *your* GitHub account.
  - **File → Download** saves it to your own computer.

  Nothing is ever saved to the author's Google or GitHub account.
- **Sharing is your decision and your responsibility.** If you share a notebook that contains your data — for example with **Share** in Colab or Google Drive, by setting access to "Anyone with the link", or by publishing it on GitHub — everyone who can open it can see the data and results in it. Making the data available in this way is your choice and your responsibility, not the author's.

- **Before sharing** a notebook, remove your data from the form and choose **Edit → Clear all outputs**, then save.
- **What the code does not do.** It makes no other network requests and writes no files.

> For unpublished or sensitive data, use the browser or Python version.

13 Differences between the versions

The calculations and results are identical. Only the interface differs:

	Browser	Python	Colab
Default base temperature	37 °C	37 °C	30 °C
Data entry	Text box or CSV file	Text box or CSV file	Two comma-separated lists
Custom base temperature	Any value	Any value	Slider, 0–100 °C
Fitting routine	Built-in Levenberg–Marquardt	SciPy `curve_fit`	SciPy `curve_fit`
Saving the figure	PNG, SVG	PNG, PDF, SVG	Right-click the image (PNG)
Saving the results	TXT file	TXT file	Copy the output
Fit range set by	Low/High fields or dragging on a plot	Low/High fields or dragging on a plot	Two sliders
Return to all data	Untick, double-click a plot, or Use all data	Untick, double-click a plot, or Use all data	Set_temperature_range = “No”
Fit range shaded on plots	Yes (on screen)	Yes (on screen)	No
Axis scales with a fit range	Same as all data	Same as all data	Same as all data
Manual axis scales	Axis scales row below the plot (applied at once)	Axis scales row below the plots (applied at once)	Form fields with sliders (applied when the cell is run)
Polynomial fit	Yes (optional)	Yes (optional)	No
Changes after a first calculation	Applied at once (except data)	Applied at once (except data)	Applied when the cell is run
Zoom and pan	No	Yes (plot toolbar)	No
Summary row of key values	Yes (incl. E_a)	No (Results tab)	No
Explanation	“How it works” window	“How it works” window	Section in the notebook
Demo notice disappears	Clear, editing data, loading a CSV	Clear, editing data, loading a CSV	When the data differs from the example
Point colour (used in fit)	Red	Red	Red
Dark mode, phones	Yes	No	Follows Colab

14 Use of AI

The author of the calculator describes the use of generative AI in its development as follows (from the repository's README):

> I wrote the core code for Q10 calculation and graph generation in Python. The graphical user interface was coded with the assistance of generative AI, based on my design. Additionally, the HTML version was generated by AI based on the Python version. I reviewed the AI-generated code and made manual corrections as needed. Furthermore, this README was created with the assistance of AI, including the design and proofreading of the document. The user manual (PDF) was entirely AI-generated based on the README.

14.1 Statement from the AI assistant

Asked whether any protected code or software had been used or reproduced in modifying the author's code, the AI assistant stated the following. The statement is reproduced as quoted in the README; in it, "I", "my" and "myself" refer to the AI assistant, and "you" and "your" to the author.

> I didn't copy or reproduce code from any protected software. Everything I changed or added was written for your project, building on your original code. There are some third-party components the files depend on, but none of them were copied into your code.
>
> **What I wrote, and what it's based on**
>
> The HTML version's fitting code: I wrote the JavaScript Levenberg–Marquardt fit myself. It's based on the published, standard algorithm, not ported from SciPy, MINPACK or any other library's source code. The same goes for the other mathematics: the least-squares and standard-error formulas, the Arrhenius regression, and the "nice number" (1-2-5) rule for axis ticks. These are textbook methods.
>
> The Python and Colab changes: these extend your own code. The example data and the overall structure come from your files.
>
> **Third-party components used, not copied**
>
> numpy, SciPy, matplotlib, pandas and Tkinter: your original code already used these, and the updated versions still import them. They're open source: numpy, SciPy and pandas use the BSD licence, matplotlib its own BSD-style licence, and Python and Tkinter the PSF licence. Importing them doesn't put any licence obligations on your code. Obligations would apply only if you redistributed the libraries themselves.
>
> Google Fonts (HTML version): the typefaces Instrument Sans and JetBrains Mono are under the SIL Open Font License. The page loads them from Google rather than including them in the file.
>
> CodeCogs (README): the formula images are generated and hosted by the CodeCogs service, not stored in your repository. If you rely on it publicly, check CodeCogs' terms of use, which I haven't verified.
>
> Colab form syntax (#@title, #@param): this is a Colab feature that your original notebook already used.
>
> **Tools that aren't in your files**
>
> To test the code, I used a browser automation tool (Playwright), ImageMagick, and SciPy's linregress for comparison. None of these are part of the delivered files.
>
> **One caveat**
>
> I can't give an absolute guarantee: my writing comes from patterns learned in training, and very common code idioms can resemble existing code. But nothing here was taken from a specific source, and the code consists of standard techniques written for your files.

> If you publish the repository, it's worth adding a LICENSE file (for example MIT or BSD-3) so others know how they may use your code. I'm not a lawyer, so for formal licensing questions it's best to consult one.

14.2 How the results were checked

Because parts of the code were written with AI assistance, the calculations were verified independently of the code that produces them:

- The browser version's own Levenberg–Marquardt fit was compared with SciPy's `curve_fit`, and the three versions with one another, using the example data with several base temperatures and fit ranges. τ_b, Q_{10}, their standard errors and R^2 agree to every printed digit (Section 3.8).
- The Arrhenius regression values (E_a, slope, intercept, their standard errors and R^2) were compared with SciPy's `linregress` and agree.
- Input checks, error messages, saving, range selection and the fixed axis scales were tested by operating the applications automatically.

As with any analysis software, check that the results are plausible for your data — for example by looking at the plots — before relying on them.

15 License

The Q_{10} calculator is released under the **MIT License**. The licence text is in the file `LICENSE` in the repository (github.com/dxda6216/q10) and is reproduced below.

```
MIT License

Copyright (c) 2026 dxda6216

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

15.1 What this means in practice

- **You may** use, copy, modify, merge, publish, distribute, sublicense and sell copies of the software, free of charge, including for commercial purposes.
- **The condition** is that the copyright notice and the permission notice are included in all copies or substantial portions of the software.
- **No warranty.** The software is provided “as is”. The authors or copyright holders are not liable for any claim, damages or other liability arising from its use. You are responsible for checking that the results are correct and suitable for your purpose.
- **Documentation.** The licence covers the software “and associated documentation files”.
- **Third-party components** that the calculator uses but does not include — Python and Tkinter, numpy, SciPy, matplotlib, pandas, the Google Fonts typefaces and Google Colab — are covered by their own licences or terms (Appendix C).

This summary is for orientation only and is not legal advice; the licence text above is authoritative.

Appendix A Formula summary

Quantity	Formula
Q_{10} model	$\tau(T) = \tau_b \cdot Q_{10}^{\frac{T_b - T}{10}}$
Frequency	$f = \frac{24}{\tau}$ (day^{-1})
Log-linear form	$\ln f = \ln \frac{24}{\tau_b} + \frac{T - T_b}{10} \ln Q_{10}$
Absolute temperature	$T_K = T(°C) + 273.15$
Arrhenius regression	$\ln f = b_0 + b_1 \cdot \frac{1000}{T_K}$
Activation energy	$E_a = -1000 R b_1$, R = 8.314462618 J mol^{-1} K^{-1}
Q_{10} ↔ E_a	$E_a = \frac{R T_1 T_2 \ln Q_{10}}{10}$ (T in K)
Residual sum of squares	SSE = $\Sigma (\tau_i - \hat{\tau}_i)^2$
Standard errors	square roots of the diagonal of $s^2 (J^T J)^{-1}$, s^2 = SSE/(n − 2)
R^2	1 − SSE/SST, SST = $\Sigma (\tau_i - \bar{\tau})^2$
Polynomial fit (optional)	$\tau(T) = a_0 + a_1 T + \ldots + a_d T^d$, d = 1–6; adjusted $R^2 = 1 - (1 - R^2)(n - 1)/(n - d - 1)$

Appendix B Glossary

Term	Meaning
Activation energy (E_a)	The energy barrier in the Arrhenius equation; here, a measure of how steeply the clock's frequency rises with temperature, in kJ/mol.
Arrhenius plot	A plot of the logarithm of a rate against 1/T (here 1000/T); straight for a process that follows the Arrhenius equation.
Base temperature (T_b)	The temperature at which the fitted period τ_b is reported. It does not affect Q_{10}.
Frequency	Number of cycles per day, 24 divided by the period in hours.
Levenberg–Marquardt algorithm	A standard method for non-linear least squares that blends the Gauss–Newton method with steepest descent.
Period (τ)	The duration of one cycle of the rhythm, in hours.
Q_{10}	Temperature coefficient: the factor by which a rate changes for a 10 °C rise in temperature.
R^2 (coefficient of determination)	The fraction of the variation in the data explained by the fitted model.
Residual	The difference between a measured value and the value predicted by the model.
Standard error (SE)	An estimate of the statistical uncertainty of a fitted parameter.
Temperature compensation	The property of circadian clocks of keeping a nearly constant period across temperatures ($Q_{10} \approx 1$).
Polynomial fit	An optional least-squares polynomial of period on temperature, describing the shape of the data without a physical model.
Adjusted R^2	R^2 corrected for the number of fitted coefficients, so that adding terms is not rewarded automatically.

Appendix C Third-party components

The calculator's own code is released under the MIT License (Section 15) and does not include code copied from other software (Section 14.1). It relies on the following components, which are used, not redistributed, by the calculator files:

Component	Used by	Licence / terms
Python, including Tkinter	Python version	Python Software Foundation License
numpy	Python, Colab	BSD 3-Clause
SciPy	Python, Colab	BSD 3-Clause
matplotlib	Python, Colab	matplotlib License (BSD-style, PSF-based)
pandas	Colab	BSD 3-Clause
IBM Plex Sans, IBM Plex Mono (via Google Fonts)	Browser version	SIL Open Font License 1.1; loaded from Google, not included in the file
Google Colab	Colab version	Google's terms of service